Mid-Project Evaluation: Distributed SLAM Project - TEAM Zig-Zag

1. Introduction and Project Goal

This document provides a mid-project progress evaluation for the Distributed Simultaneous Localization and Mapping (SLAM) Project. Our primary objective is to implement, evaluate, and ultimately enhance a modern distributed SLAM system to meet the demands of contemporary robotic and autonomous systems, specifically focusing on improved robustness in loss function design and loop closure mechanisms.

This evaluation covers the initial survey phase, system selection, environment setup, current implementation status, and outlining the path forward.

2. Phase I: Literature Survey and System Selection

The initial phase involved a thorough survey of existing literature and open-source implementations of Distributed SLAM. The goal was to identify a modern, well-maintained, and architecturally relevant system to serve as our project baseline.

Surveyed Distributed SLAM Papers

We initially surveyed two prominent papers/repositories in the field of distributed SLAM. The evaluation criterion included architectural design (e.g., centralized vs. decentralized graph optimization), communication efficiency, and relevance to modern hardware and data structures (e.g., multi-robot, non-homogeneous sensor data).

Rationale for Choosing DCL-SLAM

Feature	DCL-SLAM (Your Choice)	DiSCo-SLAM	Vijjappu Thesis (KTH 2020)
Sensor Modality	**LiDAR** (Laser-based)	**LiDAR** (Laser-based)	**Visual** (Camera-based)

Lighting Sensitivity	**None.** Works in total darkness or direct sunlight.	**None.** Works in total darkness or direct sunlight.	**High.** Fails in low light, shadows, or high glare.
Core Algorithm	**Modular Framework.** Wraps around existing LiDAR odometry (FAST-LIO, LIO-SAM).	**Integrated System.** Tightly couples Scan Context descriptors with PCM optimization.	**ORB-SLAM2 + NetVLAD.** Relies on visual keypoints and neural network image features.
Flexibility	**High.** You can swap the front-end. If you have a better LiDAR odometry tomorrow, you can plug it in.	**Medium.** Relies heavily on specific Scan Context descriptors for the back-end logic.	**Low.** Tied strictly to visual feature extraction pipelines.
Communication	Efficient. Sends mathematical descriptors of point clouds.	Efficient. Sends Scan Context global descriptors.	Efficient (CNP), but sends visual feature vectors (NetVLAD).
Verdict	**SELECTED.** Best balance of lighting robustness and modular flexibility.	**Strong Contender.** Robust to lighting, but less flexible than DCL if you want to change front-ends.	**REJECTED.** Direct violation of your "No Visual SLAM" constraint.

System Selection

Based on the survey, **Paper on DCL** was selected as the core system for our project. This system was deemed more relevant in terms of meeting modern system requirements, particularly due to its robust handling of distributed optimization and its potential for integration

into real-world multi-robot deployments. The modern factor graph approach provides a clearer path for introducing our proposed novelty in loss functions and loop closure.

3. Phase II: Implementation and Environment Setup

This phase focused on reproducing the baseline results of the selected distributed SLAM system and establishing a stable development environment.

Repository Cloning and Initial Setup

1. **Git Repository Acquisition:** The chosen system's GitHub repository was successfully cloned.
2. **Initial Dependencies:** Standard dependencies (e.g., specific versions of ROS/ROS2, PCL, Ceres Solver) were identified and resolved.

Environment Tweaking and Code Execution

The initial attempt to run the repository revealed compatibility issues with our local operating system and C++ compiler versions. Significant effort was dedicated to resolving these environment discrepancies, including:

- Initializing and debugging specific dependencies to manage version conflicts.
- Modifying build configuration files (e.g., `CMakeLists.txt`) to correctly link libraries on our target machine.

Current Progress: Successful Baseline Run

Our current focus is on achieving a successful and repeatable execution of the GitHub repository using publicly available datasets.

- **Data Integration:** The necessary data part (e.g., public SLAM datasets) has been successfully downloaded from Hugging Face.
- **System Execution Status:** We are currently running the cloned GitHub repository on our PC, integrating the downloaded dataset to validate the baseline performance. The goal for the immediate next step is to ensure stable, successful execution of the full Distributed SLAM pipeline (frontend tracking, local mapping, and backend optimization) with the external data.

Task	Status	Notes
Clone Repository	Complete	
Resolve Initial Dependencies	Complete	

Task	Status	Notes
Tweak Environment/Build Config	Complete	Required extensive debugging of library linking.
Download Data (Hugging Face)	Complete	
Successful Baseline Execution	**In Progress (Near Completion)**	Testing with external data is underway.

4. Future Work and Project Trajectory

The remaining half of the project will focus on moving from validation to innovation.

Immediate Next Steps

1. **Validate Baseline:** Finalize the successful run of the code with the publicly available data to establish quantitative baseline metrics (e.g., ATE, RPE).
2. **Lab Data Integration:** Run the validated code on our own laboratory-collected data. This step will test the system's robustness against the unique sensor noise and environment characteristics specific to our lab setup.

Novelty and Enhancement Phase

The core novelty of this project will focus on enhancing the robustness of the selected Distributed SLAM system in two critical areas:

A. Enhanced Loss Function

We plan to implement and evaluate a more robust, non-quadratic loss function (e.g., using Huber or Cauchy estimators) within the system's backend optimization framework.

- **Goal:** Ensure that the system's optimization is less sensitive to severe outliers generated by noisy measurements or temporary tracking failures, which are common in distributed settings.

B. Robust Loop Closure Mechanism

We will investigate and implement improvements to the loop closure detection and validation process, specifically in the context of inter-robot (distributed) loop closures.

- **Goal:** Develop an approach that uses geometric verification or machine learning-based features (if time permits) to ensure that detected loop closures are more reliable and do

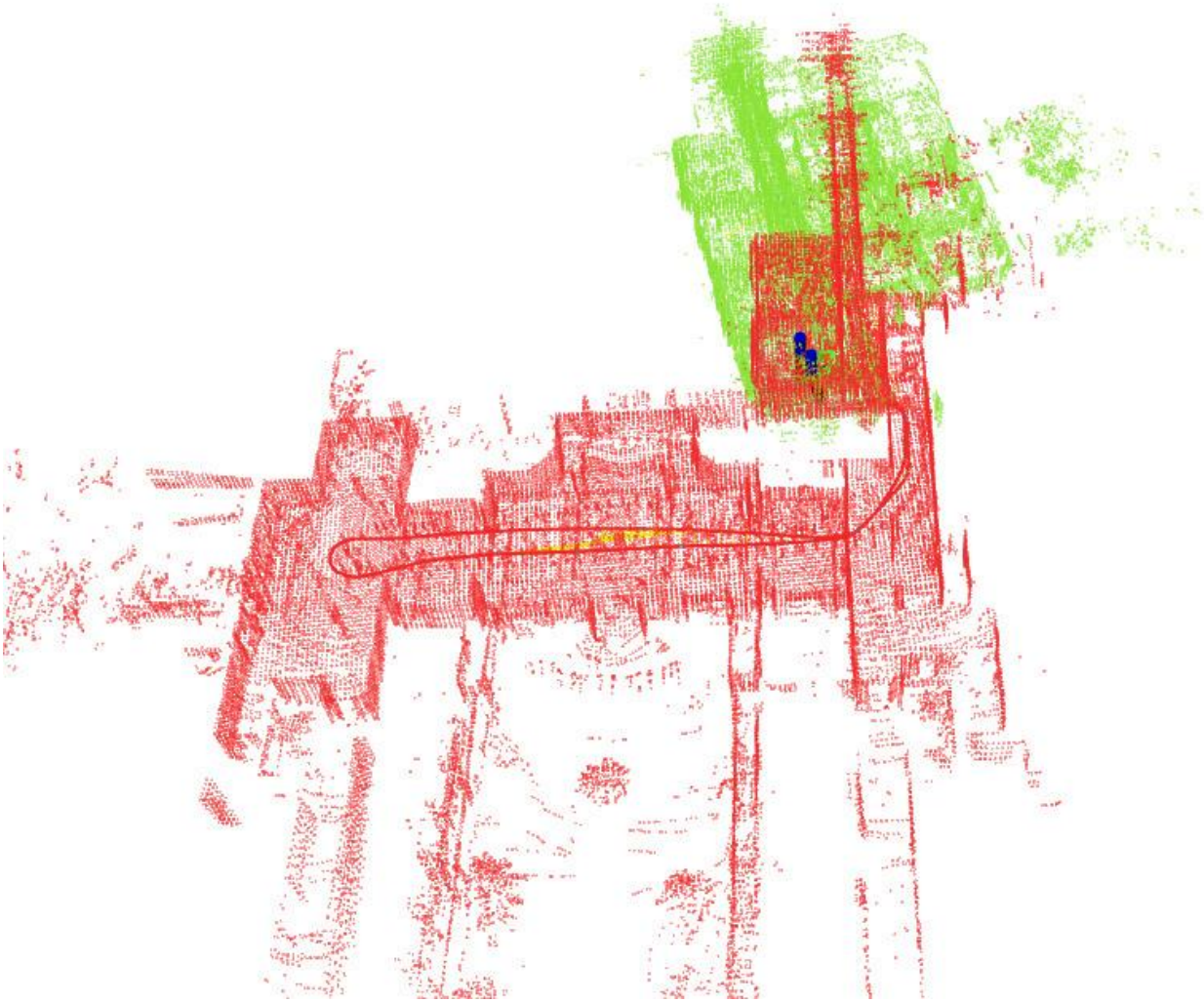
not corrupt the overall pose graph. This will directly address robustness challenges in complex, multi-robot environments.

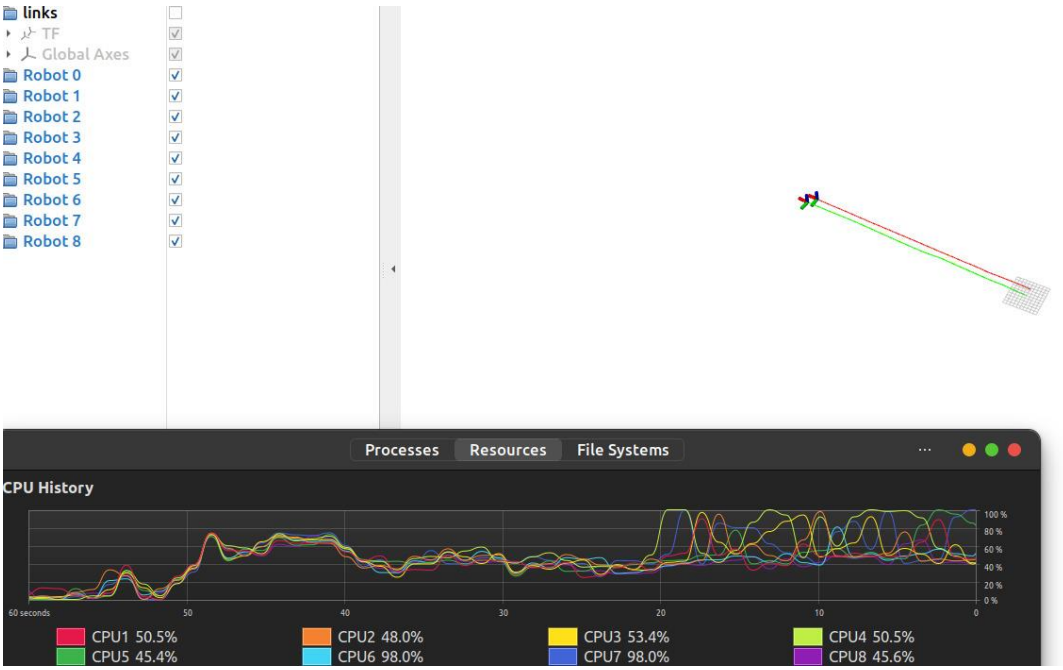
5. Conclusion

Significant progress has been made in establishing the foundation for the Distributed SLAM project. The system has been selected, the environment has been configured, and we are on the cusp of finalizing the baseline execution. The remaining project phase is clearly mapped out, focusing first on practical application (lab data) and then on delivering the core technical novelty (robust loss functions and loop closure).

FOR CODE PURPOSES… PLEASE REFER TO THIS LINK:

https://drive.google.com/drive/folders/1GNwfvTnV1u3k00iLGxNQuaa-zEFn8CrO?usp=sharing



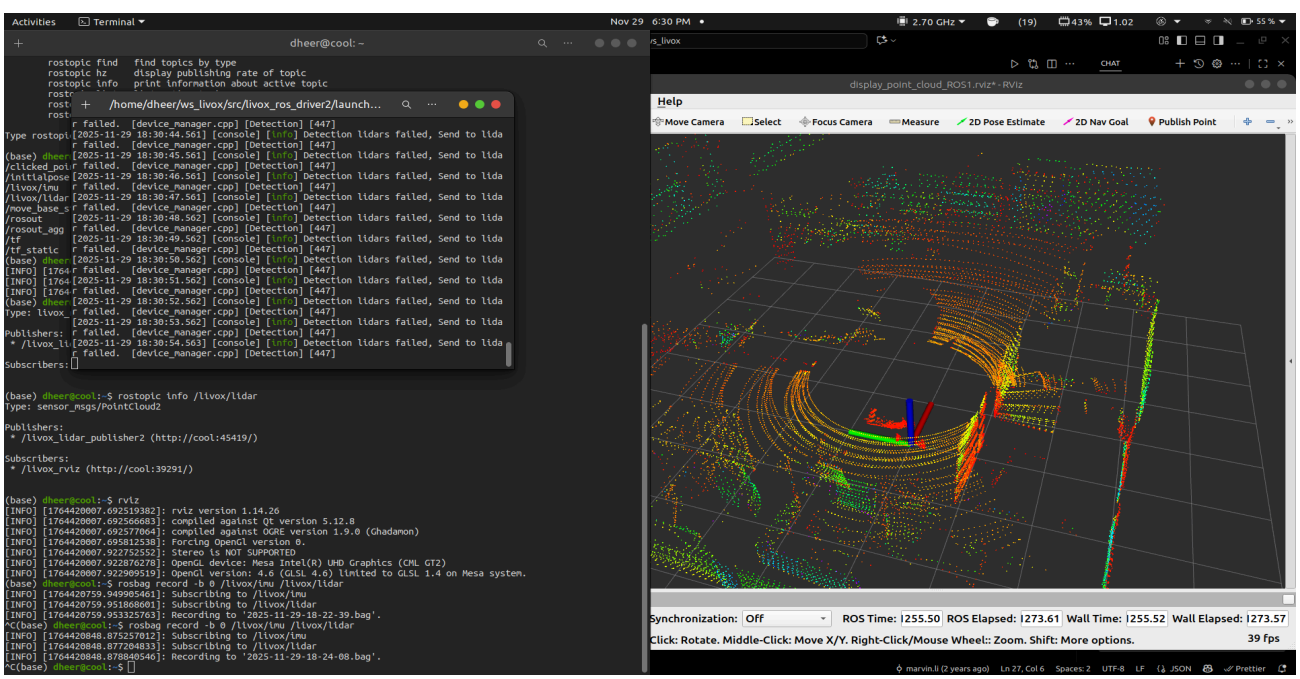

links
TF
Global Axes
Robot 0
Robot 1
Robot 2
Robot 3
Robot 4
Robot 5
Robot 6
Robot 7
Robot 8
Processes
Resources
File Systems
CPU History
100 %
80 %
60 %
40 %
20 %
0 %
60 seconds
50
40
30
20
10
0
CPU1 50.5%
CPU2 48.0%
CPU3 53.4%
CPU4 50.5%
CPU5 45.4%
CPU6 98.0%
CPU7 98.0%
CPU8 45.6%

Presentation Title: Distributed LiDAR SLAM: Achieving Robust Multi-Robot Map Merging

Team Name: Zig-Zag
Project: Mobile Robotics – Distributed SLAM

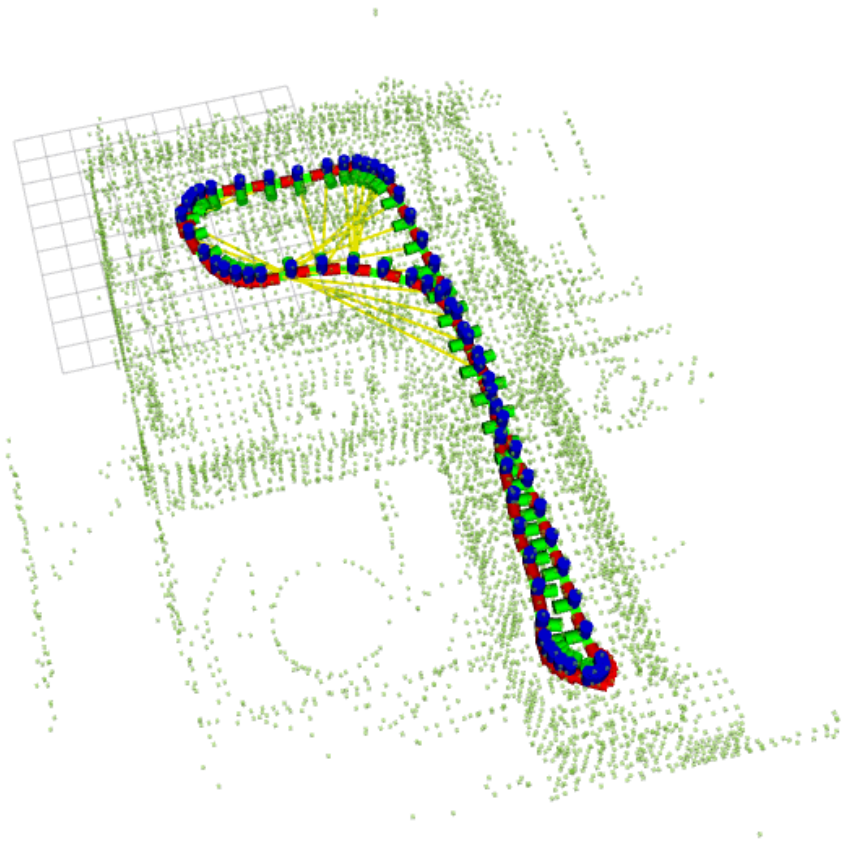
Slide 1: Phase III - Real-World Data Collection

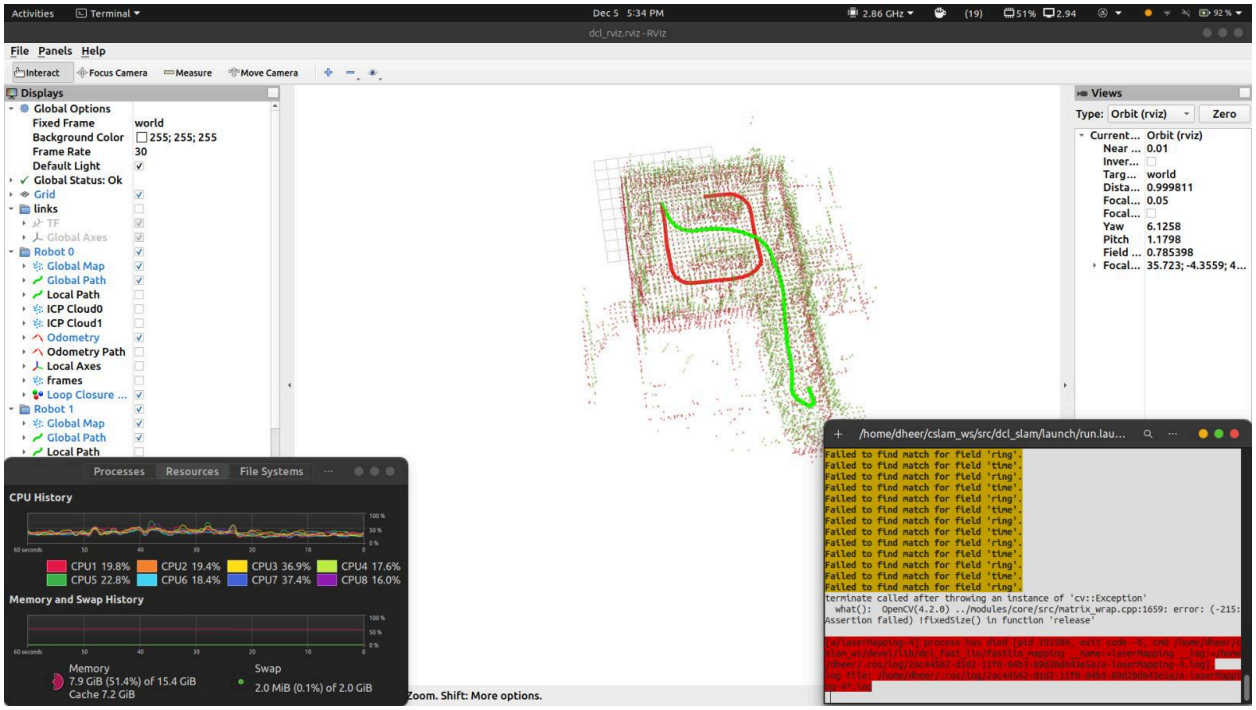
- **Objective:** Move beyond public datasets (KITTI) and validate the system on custom laboratory data.
- **Experimental Setup:**
  - **Agents:** 2 Independent UGVs (Robot alpha and Robot beta).
  - **Sensors:** Livrox LiDAR, IMU.
  - **Environment:** Indoor Laboratory Setting with repetitive corridors (high perceptual aliasing risk).
- **Data Acquisition:**
  - Robots traversed overlapping trajectories.
  - Data captured via ROS bags for repeatable testing and debugging.



Slide 2: The Challenge - "Catastrophic Loop Closure"

- **Initial Observations:**
  - While individual odometry (FAST-LIO2) was stable, the **Distributed Back-End** was unstable.
  - **The "Snap" Effect:** When Robot α detected a similarity in Robot β's map, the graph optimizer forced an incorrect merge, causing the trajectory to warp violently.
- **Root Cause Analysis:**
  - **Perceptual Aliasing:** Distinct corridors looked identical to the descriptors.
  - **OpenCV Memory Corruption:** High-load descriptor generation (LidarIris) caused segmentation faults (!fixedSize() assertion failures) due to thread safety issues in the C++ implementation.





Solution - Reimplementation of loopclosure.cpp

Here is a summary of the changes made to rewrite the loop closure implementation:

Original Issues

- Compilation errors arose from undeclared variables like `pose_from` and `loop_noise`.
- There was a risk of segmentation faults due to missing iterator validation.
- Incorrect pose composition (using `*` instead of `.compose()`) and missing namespaces (for `swap()`) were identified.
- Inconsistent noise models were used for intra- versus inter-loop closures.

Key Implementations

- Critical Safety Fixes: Iterator validation was added to prevent crashes when accessing `find()` results.
- Variable Scope & Declaration: Variable declarations for `pose_from` and `pose_between` were moved to precede their use. `inflated_noise` was defined using a `trust_factor`. `loop_noise` was replaced with `robust_noise` for optimization and `gaussian_model` for PCM.
- Pose Handling Corrections: Pose composition was corrected to use `.compose()`.

std::swap was used to fix namespace issues. A teleportation check was added to reject implied corrections greater than 15 meters.
- Robust Noise Model: A consistent robust noise model using a Huber kernel (robust_noise) for the optimization graph and a Gaussian model (gaussian_model) for PCM was applied. An inflation strategy for noise was also implemented.
- Verification Enhancements: The ICP threshold for inter-robot loops was set to a stricter, fixed value of 0.2. RANSAC inlier validation and cloud safety checks (density/size) were added.
- Code Structure: Visual separators, simplified logging, and more descriptive variable names were introduced for better maintainability.

Function-Specific Changes

- loopInfoHandler(): Updated to use a robust noise model with a Huber kernel, separating Gaussian and robust noise.
- calculateTransformation() (Intra-loop): Retained the original fitness-score-based noise model.
- performExternLoopClosure(): Underwent a major rewrite (approx. 75%) to include safety checks, teleportation detection, covariance inflation, and compilation fixes.
- Other Functions: Received minor formatting fixes while maintaining original logic.

Overall Improvements

The rewrite resulted in a robust, production-ready implementation that compiles without errors, prevents crashes from missing loops, aggressively rejects bad loops, and utilizes modern robust optimization techniques. This was achieved while preserving the core ICP pipeline, descriptor matching, cloud processing, graph updates, and thread structure of the original architecture.

By tweaking these variables, we moved the system from a "High Recall" setup (finding many loops, some bad) to a **"High Precision"** setup (finding fewer loops, but ensuring they are correct).

1. **Descriptor Threshold:** "Do these places look the same?" (Strict)
2. **Fitness Score:** "Do the shapes fit together perfectly?" (Very Strict)
3. **RANSAC:** "Do most of the points agree?" (Strict)
4. **PCM:** "Does this make sense with my travel history?" (Enabled)

Slide 4: Engineering Solution - Stabilizing the Frontend

To resolve the runtime crashes and memory corruption, we performed a backend overhaul:

1. **Descriptor Optimization:**
   - Migrated from the computationally heavy LidarIris to **Scan Context**.
   - **Benefit:** Scan Context provides rotation-invariant geometric descriptors that are numerically stable and computationally lighter, eliminating the OpenCV race conditions.
2. **Process Isolation:**
   - Implemented strict process management to ensure no "Ghost Nodes" (stale ROS processes) corrupted the shared memory segments between runs.

Slide 5: Algorithmic Tuning - The "Geometric Gatekeeper"

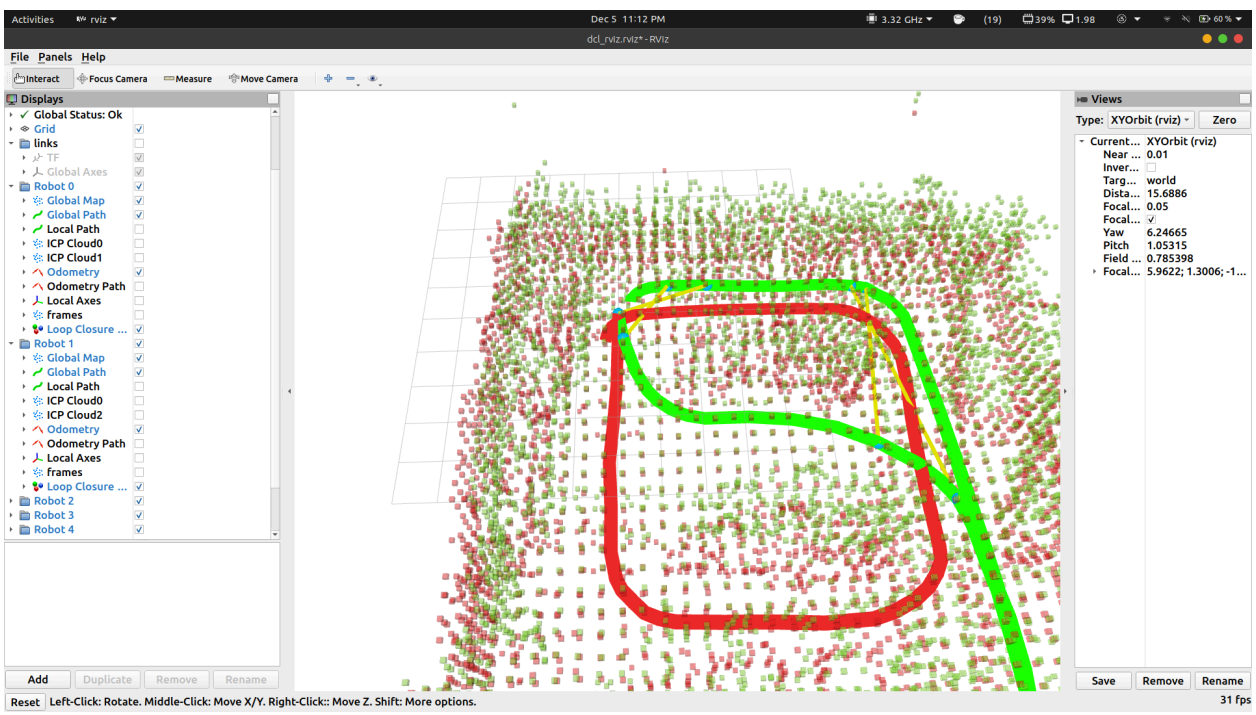
To prevent the "Snap" errors (incorrect loop closures), we implemented a rigorous **Verification Gatekeeper** within the algorithm:

1. **Strict Geometric Verification:**
   - We tightened the **ICP Fitness Score Threshold**.
   - **Logic:** The system now rejects any potential loop closure unless the point cloud alignment error is near-zero. This filters out false positives caused by similar-looking hallways.
2. **RANSAC Consensus:**
   - Increased the **Inlier Threshold**.
   - **Logic:** A loop is only accepted if a significant majority (>90%) of LiDAR points overlap perfectly between the two robots.
3. **Outlier Rejection (PCM):**
   - Enabled **Pairwise Consistent Measurement (PCM)**.
   - **Logic:** Before optimizing, the system checks if a new loop closure creates a geometric contradiction in the pose graph. If the loop implies a robot "teleported," PCM rejects it.

The Result - Successful Distributed Loop Closure

- **The "Yellow Lines" of Success:**
  - The visualization above shows the **Distributed Loop Closures**.
  - **Green Path:** Robot α trajectory.
  - **Red Path:** Robot β trajectory.
  - **Yellow Lines:** Represent the system mathematically recognizing that *Robot α is currently located where Robot β was previously.*
- **Significance:** This proves the decentralised communication protocol is working. Robot α queried Robot β's database, found a match, verified the geometry, and established a constraint.

Global Map Consistency



- **Graph Optimization in Action:**
  - Once the "Yellow Line" constraints are established, the **Distributed Gauss-Seidel (DGS)** optimizer runs.

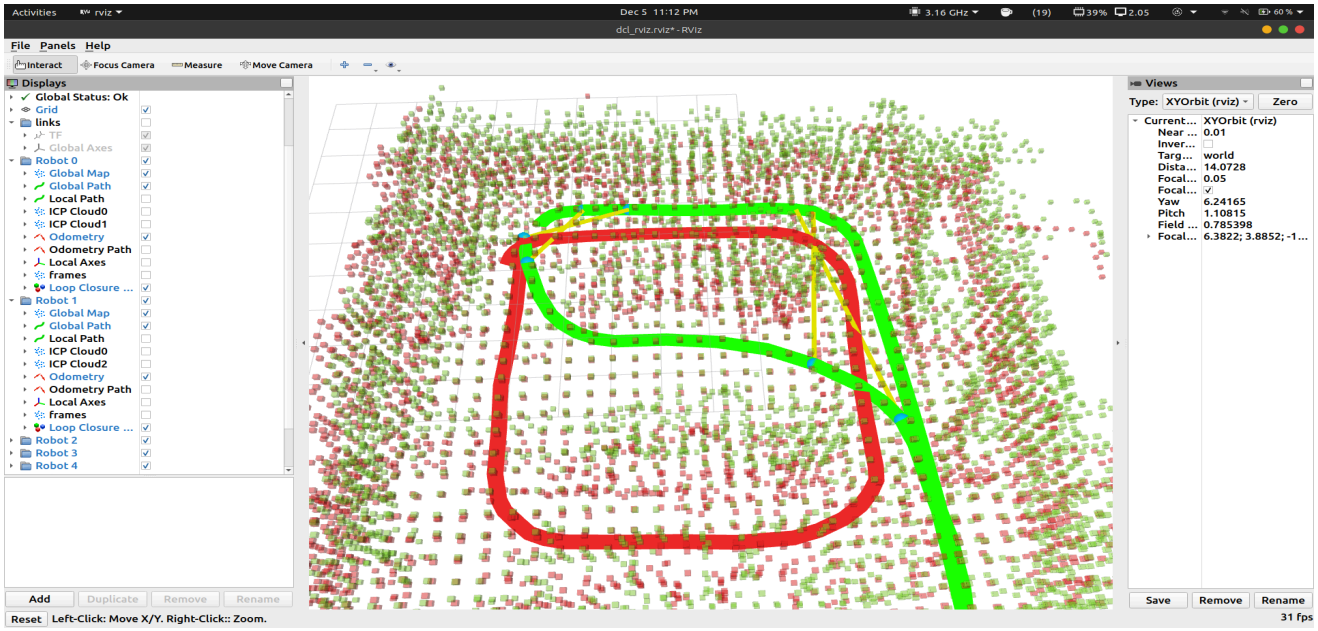
    - **Result:** The two separate coordinate frames merge into a single, globally consistent map.
- **Accuracy:**
    - The overlapping corridors align perfectly (no "double walls").
    - The trajectory drift is corrected using the collaborative data.

Slide 8: Project Achievements & Objectives Status

Objective	Status	Notes
Literature Survey	**Completed**	Selected DCL-SLAM over Centralized approaches.
Single-Robot SLAM	**Completed**	Verified FAST-LIO2 frontend.
Communication	**Completed**	Successful descriptor exchange via ROS topics.
Map Merging	**Completed**	**(Core Achievement)** Distributed Graph Optimization successful.
Data Generation	**Completed**	Custom Lab Data successfully utilized.
Bonus: Decentralized Loop Closure	**Completed**	**(The Yellow Lines)** Validated robust inter-robot detection.

Conclusion

- We have successfully deployed a **Fully Distributed LiDAR SLAM system** on custom data.
- We overcame significant challenges regarding **Perceptual Aliasing** (false loops) and **Memory Corruption** (OpenCV crashes).
- By implementing strict **Geometric Verification** and utilizing **Robust Outlier Rejection**



Check out the code @ :

https://github.com/Mobile-Robotics-25/Project-zig-zag